NORTEX

Sprachreferenz

Version 0.1.0

Inhaltsverzeichnis

Kapitel 1 – Einführung 5
- Was ist NORTEX? 5
- Ziele 5
- Eigenschaften (belegt) 5
- Abgrenzung 5
- Einsatzmöglichkeiten (aktuell realistisch) 5
- Technische Voraussetzungen 5

Kapitel 2 – Installation und Einrichtung 6

Kapitel 3 – Grundlagen der Syntax 7
- Aufbau eines Programms 7
- Anweisungen, Ausdrücke, Blöcke 7
- Kommentare 7
- Bezeichner 8
- Schlüsselwörter 8
- Typische Fehler 8
- Namenskonventionen (Empfehlung) 8
- Geplant (Zielentwurf) 8

Kapitel 4 – Datentypen und Variablen 9
- Primitive Typen 9
- Literale 9
- Variablen 10
- Typinferenz 10
- Typkonvertierungen 10
- Gültigkeitsbereiche 10
- Einschränkungen 11
- Geplant (Zielentwurf, nicht ausführbar) 11

Kapitel 5 – Operatoren 12
- Übersicht 12
- Rangfolge (hoch -> niedrig), Assoziativität 12
- Verhalten 12
- Typkompatibilität 13
- Häufige Fehler 13
- Operatorüberladung 13

Kapitel 6 – Kontrollstrukturen 14
- if / else 14
- while 14
- for 14
- break und continue 14
- return 14
- Ternär 15
- Weitere Konstrukte (nur Frontend) 15

Kapitel 7 – Funktionen und Methoden 16
- Funktionsdefinition 16
- Aufruf, Rekursion 16

Rückgabewerte 16
Lokale Variablen 16
Methoden und Sichtbarkeit (nur Frontend) 16
Einschränkungen 17
Entwurfsregeln und Frontend-Stand 17
Kapitel 8 – Structs und Datenstrukturen 18
Struct-Definition 18
Initialisierung und Zugriff 18
Klassen mit Methoden 18
Weitere Deklarationen 18
Arrays und Sammlungen 18
Verschachtelung 19
Klassen, Vererbung, Generics, Collections (Zielentwurf) 19
Kapitel 9 – Speicherverwaltung 20
Tatsächlicher Stand 20
Sprachziel: Ownership 20
Sicherheitsmechanismen (implementiert) 20
Lebensdauer 20
Runtime 20
Geplantes Speichermodell (Entwurf) 20
Kapitel 10 – Ein- und Ausgabe 22
Konsolenausgabe 22
Nicht unterstützt (jeweils NX2001/NX2002) 22
String-Konkatenation 22
Ergebniswerte als Ausgabekanal 23
Standardbibliothek 23
Strings (Zielentwurf) 23
Kapitel 11 – Fehlermeldungen und Debugging 24
Format 24
Fehlerklassen mit tatsächlich beobachteten Codes 24
Laufzeitfehler 24
Bekannter Compilerfehler 25
Strategien 25
Fehlerbehandlung (Entwurf und Frontend-Stand) 25
Asynchronität (geplant) 25
Kapitel 12 – Kompilierung und technische Funktionsweise 26
Pipeline 26
Kommandos 26
Eigenschaften des Backends 26
Hinweis zu älteren Dokumenten 27
ABI (Zielentwurf) 27
Paketformat .ntx (geplant) 27
Kapitel 13 – Praktische Beispiele 28
1. Hello World – 01_hello.nx 28

2. Variablen und Berechnungen – 02_variables.nx 28
3. Bedingungen – 03_conditions.nx 28
4. Schleifen – 04_loops.nx 29
5. Funktionen und Rekursion – 05_functions.nx 29
6. Vollständige Anwendung: Primzahlen – 06_primes.nx 29
7. Structs – 07_struct_frontend.nx (nur check) 30
8. Methoden – 08_class_frontend.nx (nur check) 30
9. Heap-Allokation 30
Kapitel 14 – Sprachreferenz 31
Schlüsselwörter 31
Token 31
Operatoren und Symbole 31
Grammatikalische Konstruktionen (nativ) 31
Unterstützte Anweisungen 32
Built-in-Funktionen 32
Diagnosecodes (beobachtet) 32
Relevante Einschränkungen 32
Kapitel 15 – FAQ und Best Practices 33
Häufige Anfängerfehler 33
FAQ 33
Empfohlene Praktiken 33
Performance 33
Größere Projekte 33
Offene Entscheidungen (Projektinhaber) 34

Kapitel 1 – Einführung

<- Inhaltsverzeichnis · Weiter: Kapitel 2

Was ist NORTEX?

NORTEX ist eine statisch und streng typisierte, kompilierte Programmiersprache mit C#-ähnlicher Deklarationssyntax. Der Compiler nortexc ist in Rust geschrieben und erzeugt über Cranelift native Windows-x64-Programme (.exe).

```
public int main() {
    return 42;
}
```

Ziele

Laut Projektdokumentation: Speichersicherheit ohne Garbage Collector (Ownership/Borrowing), explizite Veränderlichkeit, stabile Diagnosecodes (NXnnnn) und ein kompilierter Ausführungspfad. Davon sind Veränderlichkeitsregeln und Diagnosen implementiert; Ownership-Prüfung ist teilweise vorhanden (siehe Kapitel 9).

Eigenschaften (belegt)

- Bezeichner und Schlüsselwörter sind case-sensitive; Bezeichner sind ASCII-only.
- Bindungen sind standardmäßig unveränderlich; mut macht sie veränderlich.
- Kein null in sicherem Code.
- Overflow-Prüfung bei Ganzzahlarithmetik standardmäßig aktiv.
- Deutsche Compilermeldungen mit stabilen Codes.

Abgrenzung

NORTEX sieht C#-ähnlich aus, ist aber eigenständig: Top-Level-Funktionen ohne Klasse, mut-Bindungen, Ownership-Konzepte und ein eigener, deutlich kleinerer Funktionsumfang. Syntax anderer Sprachen darf nicht angenommen werden (z. B. keine Casts, kein do-while, kein let).

Einsatzmöglichkeiten (aktuell realistisch)

Kleine Konsolenprogramme mit ganzzahliger Logik, Compiler-/Sprachexperimente, Lernprojekt. Allgemeine Anwendungsentwicklung ist erst nach Umsetzung von Strings, Structs, Heap und Standardbibliothek möglich (siehe README).

Technische Voraussetzungen

Windows 10/11 x64, Rust-Toolchain (Edition 2024, Cargo >= 1.88) und die MSVC-Buildtools (Linker link.exe). Siehe Kapitel 2.

Kapitel 2 – Installation und Einrichtung

<- Kapitel 1 · Inhalt · Weiter: Kapitel 3

Voraussetzungen (Windows x64, Rust >= 1.88, MSVC Build Tools), Build-Befehle, erstes Programm und das Manifest nortex.toml stehen zentral in der Installationsanleitung.

Kurzfassung:

```
cargo build --release -p nortex-compiler
nortexc build hello.nx -o hello.exe
```

Hinweis: using util; löst eigene .nx-Dateien auf (flacher Namensraum), siehe Ist-Spezifikation.

Kapitel 3 – Grundlagen der Syntax

<- Kapitel 2 · Inhalt · Weiter: Kapitel 4

Aufbau eines Programms

Eine Quelldatei (.nx) besteht aus optionalen using-/namespace-Zeilen und Deklarationen. Funktionen dürfen direkt auf oberster Ebene stehen. Der native Einstiegspunkt ist genau eine parameterlose Funktion public int main(); ihr Rückgabewert wird zum Exitcode.

```
using System;

public int main() {
    Console.writeLine("Hallo");
    return 0;
}
```

void main(), main(int a) und bool main() werden beim Build mit „Windows-Programme benötigen derzeit genau eine parameterlose int-main-Funktion“ abgelehnt; ohne main scheitert der Linker (LNK2019).

using System; ist für Console im getesteten Pfad nicht nötig, aber üblich. using mit unbekanntem Namespace (using Foo.Bar;) ergibt NX1003.

Anweisungen, Ausdrücke, Blöcke

- Eine Anweisung endet mit ; (Deklaration, Zuweisung, Aufruf, return, break, continue).
- Ein Ausdruck liefert einen Wert (a * b, f(x), c ? 1 : 2).
- Ein Block { ... } gruppiert Anweisungen und bildet einen eigenen Gültigkeitsbereich.

```
public int main() {
    int a = 1;            // Deklaration
    mut int b = a + 2;    // Ausdruck als Initialisierung
    if (b > 2) {          // Block
        b += 1;
    }
    return b;
}
```

Fehlendes Semikolon: error[NX0100]: Nach einer Variablendeklaration wird ';' erwartet. Nach Blöcken (}) steht kein ;.

Kommentare

```
// Zeilenkommentar
/* Blockkommentar */
```

Kommentare dürfen Unicode enthalten.

Bezeichner

[A-Za-z_][A-Za-z0-9_]*, nur ASCII. Groß-/Kleinschreibung ist signifikant (name, Name, NAME sind verschieden). Stilwarnungen NX0004: Konstanten sollen UPPER_SNAKE_CASE, Variablen/Member camelCase sein (Warnung, kein Fehler).

Schlüsselwörter

Alle reservierten Wörter stehen in Kapitel 14. get/set sind normale Bezeichner.

Typische Fehler

Fehler	Meldung
Unbekanntes Zeichen (@)	NX0100 Unbekanntes Zeichen '@'
Reserviertes let	NX0100 'let' ist reserviert, aber in NORTEX nicht unterstützt
Unbekannter Bezeichner	NX1004
Doppelte Deklaration im selben Scope	NX1001

Namenskonventionen (Empfehlung)

Bezeichner bestehen aus ASCII-Buchstaben, Ziffern und _; Unicode ist nur in Kommentaren und String-/Char-Literalen erlaubt. Typen PascalCase (ohne I-Präfix bei Interfaces), Methoden und Variablen camelCase, Konstanten UPPER_SNAKE_CASE, Modul-/Dateinamen snake_case. Zwei-Buchstaben-Akronyme bleiben groß (IOStream, getID), längere werden wie Wörter behandelt (XmlParser). Private Felder tragen keine führenden Unterstriche. Verstöße sind Warnungen (NX0004), keine Fehler.

Geplant (Zielentwurf)

- using importiert nur NORTEX-Namespaces; es gibt keine Verbindung zu .NET.
- Sichtbarkeiten public, private, protected, internal werden geparst; ihre Durchsetzung über Modulgrenzen fehlt (siehe Kapitel 14).
- Das Paketformat .ntx ist Teil des Entwurfs, siehe Kapitel 12.

Kapitel 4 – Datentypen und Variablen

<- Kapitel 3 · Inhalt · Weiter: Kapitel 5

Primitive Typen

Typ | Breite | Nativ (Build)

int | 32 Bit, vorzeichenbehaftet | ja

bool | true/false | ja

uint, short, ulong | 32/16/64 Bit | Locals und Parameter/Rückgabe ja (uint-Funktion getestet)

long | 64 Bit | Parameter/Rückgabe ja; Locals mit Division/Vergleich können im Codegen abstürzen (Cranelift-Panic, bekannt)

sbyte, byte, ushort, i128, u128, nint, nuint, isize, usize | siehe diese Seite (Abschnitt „Geplant“) | Frontend; nicht einzeln nativ geprüft

float, double, decimal | IEEE-754 32/64 bzw. 128 Bit | nein (NX2001)

char | Unicode-Skalar | nein (NX2002)

string | UTF-8-Slice (Datenzeiger + Byte-Länge) | ja, als Literal, lokale Variable, interner Parameter/Rückgabewert und Console-Argument

void | kein Wert | nur als Rückgabetyp

Literale

String-Literale und ihre 16-Byte-Slice-Deskriptoren liegen schreibgeschützt im COFF-Datenbereich. Die Länge zählt UTF-8-Bytes.

- Konkatenation: a + b ruft nortex_string_concat der Runtime auf. Sie reserviert per HeapAlloc einen Block aus Deskriptor, Bytes und abschließendem NUL und liefert den neuen Slice. Schlägt die Allokation fehl, bricht das Programm mit einem Trap ab.
- Globals: const string und mut string sind auf Top-Level erlaubt. Der Initialisierer ist ein Stringliteral oder eine +-Verkettung von Literalen (zur Compilezeit gefaltet). Das Global ist ein Zeiger auf einen Deskriptor in .rdata; mut string ersetzt nur den Slice.
- extern "C": String-Argumente werden als char* (Datenzeiger) übergeben. Alle Strings sind NUL-terminiert; ein eingebettetes \0 beendet den String aus C-Sicht früher. Externe Funktionen dürfen keinen string zurückgeben.
- Vergleich: == und != rufen nortex_string_eq (Längen- und Byte-Vergleich) auf und liefern bool. Ordnungsoperatoren (, …) auf Strings sind nicht implementiert.
- Drop und Move: Heap-Strings (Ergebnisse von +) gehören genau einer Variable. Beim Scope-Ende, bei return und vor dem Überschreiben ruft der Codegen nortex_string_free (HeapFree) auf, außer der Wert wurde verschoben. Zuweisung, Initialisierung, Rückgabe und Übergabe an interne Funktionen verschieben den Quellstring; jede weitere Verwendung meldet NX1202. Konkatenation, Vergleich, Console und extern "C"-Aufrufe leihen nur. Statische Literale und const string werden nie freigegeben.
- Grenzen: Beim Verschieben eines mut string-Globals (string b = name;, Rückgabe, Argument, Ternary-Zweig) wird der Wert kopiert (name + ""); das Global behält seinen Puffer. Beim Neuzuweisen wird der alte Heap-Puffer freigegeben.

```
int a = 42;
int h = 0xFF;          // hexadezimal
int b = 0b1010;        // binär
```

```
double d = 1.5e3;      // Exponent (Frontend)
float f = 2.5f;        // Suffix f nötig; 2.5 ist double
char c = 'x';
string s = "text";
bool t = true;
```

Literale werden gegen den Zieltyp geprüft: int a = 3000000000; -> NX1102 … liegt außerhalb des Wertebereichs von 'int'. Integer-Suffixe wie 5L lexen, scheitern aber im nativen Lowering (NX2002). Escapes in Strings: \n \r \t \0 \" \\; unbekannte (\q) -> NX0100.

Variablen

```
int a = 1;          // unveränderlich
mut int b = 2;      // veränderlich
const int LIMIT = 3;
```

- Unveränderlich ist Standard. a = 2; -> NX1200 Der unveränderliche Wert 'a' darf nicht neu zugewiesen werden.
- Globale Werte dürfen auch auf oberster Ebene stehen. Sie benötigen einen compile-time auswertbaren Initialisierer; mut erlaubt spätere Änderungen. Beispiel: public const int MAX_PLAYERS = 100; und mut int playerCount = 0;.
- Globale Ganzzahl- und bool-Werte werden nativ in schreibgeschützten bzw. beschreibbaren COFF-Datensektionen abgelegt. Andere global deklarierbare Typen werden im Frontend geprüft, sind im nativen MIR-Subset aber noch nicht verfügbar (NX2002).
- Auch Parameter und Schleifenvariablen sind unveränderlich; für for muss mut int i verwendet werden.
- const verlangt UPPER_SNAKE_CASE (sonst Warnung NX0004).
- Jede Variable braucht Typ und Initialisierer. let ist reserviert und nicht nutzbar. readonly int a = 1; ist als lokale Deklaration ungültig.
- null ist verboten (NX1102).

Typinferenz

var c = a * b; besteht check, wird aber im Build abgelehnt: „Variablentyp-Inferenz wird in dieser Lowering-Stufe nicht unterstützt“ (NX2001). Im nativen Code immer explizite Typen verwenden.

Typkonvertierungen

Es gibt keine implizite Konvertierung zwischen Ganzzahltypen: uint + int -> NX1102 Die Operanden von '+' müssen denselben numerischen Typ besitzen; return eines byte aus int-Funktion -> NX1102. Eine funktionierende Cast-Syntax ist nicht vorhanden ((int)x und x as int werden nicht geparst).

Gültigkeitsbereiche

Ein Block {} bildet einen Scope. Variablen sind ab der Deklaration bis zum Blockende sichtbar; Verwendung außerhalb -> NX1004. Doppelte Namen im selben Scope -> NX1001. Ein verschachtelter Scope kann einen äußeren Namen im getesteten Fall nicht neu zuweisen, wenn dieser unveränderlich ist (NX1200).

```
public int main() {
    mut int a = 1;
```

```
    if (a == 1) {
        int b = 5;
        a = b;          // ok: a ist mut
    }
    return a;           // 5; b ist hier nicht sichtbar
}
```

Einschränkungen

Nativ nur skalare Ganzzahl-/Bool-Werte. Strings, Gleitkomma, Arrays, Structs und Klassen sind noch nicht ausführbar.

Geplant (Zielentwurf, nicht ausführbar)

Primitive Typnamen sind case-sensitive und haben keine Aliasse (Int32, String, Bool, Unit, Never sind ungültig). Implizite verlustbehaftete numerische Konvertierungen sind ausgeschlossen. char ist ein 32-Bit-Unicode-Skalarwert; string ist UTF-8, length() zählt Bytes. void ist kein speicherbarer Werttyp; der divergierende Typ heißt !. nint/nuint/isize/usize sind pointerbreit (Windows x64/ARM64: 64 Bit).

Typinferenz: var x = …; leitet den Typ aus dem Initialisierer ab (Ist: unveränderlich, mut var für veränderbar). Eine globale Inferenz über Parameter, Rückgaben und Generics ist nicht implementiert; mehrdeutige Inferenz soll ein Fehler sein. const verlangt einen Compile-Time-Wert.

Decimal128 (Entwurf): decimal soll ein eigenes 128-Bit-Festkommaformat sein: 1 Vorzeichenbit, 7 Bit Scale (0–38), 120-Bit-Koeffizient (0 bis 10³?-1); Wert (-1)^sign x coefficient / 10^scale, Rundung nearest, ties to even; Überlauf/Division durch null sollen ArithmeticException auslösen. Keine stille Umwandlung in double. Nur das Format ist spezifiziert, es gibt keine Arithmetik.

Zusammengesetzte Typen (Entwurf): T[] (dynamisch), [T; N] (fest), Tupel (T1, T2, …) mit mindestens zwei Elementen, struct, class, record (unveränderlich, strukturelle Gleichheit), enum (Summentyp), interface, trait. Option (Some/None) und Result (Ok/Err); null gibt es im sicheren Modell nicht.

Weitere Typkonzepte (Entwurf, nur als Namen bekannt):

Typ	Vorgesehene Semantik
object	explizites Type-Erasure/Boxing, kein universeller Basistyp
dynamic	dynamische Auflösung mit Laufzeitprüfung (DynamicDispatchException)
&T, &mut T	geteilter bzw. exklusiver Borrow
*const T, *mut T	Raw-Pointer, Dereferenzierung nur in unsafe
Box	eindeutiger Heap-Owner
Rc, Arc	geteiltes Eigentum (nicht threadsicher / atomar)
Cell, RefCell	interne Veränderlichkeit (RefCell mit Laufzeitprüfung)
Mutex, RwLock	Synchronisation; Send/Sync markieren Thread-Sicherheit
Span	nicht besitzende Sicht auf zusammenhängenden Speicher
Memory	besitzender Runtime-Handle
Func, Action	aufrufbare Werte; Captures folgen Move-/Borrow-Regeln
Future, Task	Berechnung bzw. eingeplante Ausführung, siehe Kapitel 11

Die Registrierung eines Typnamens im Compiler belegt keine Implementierung.

Kapitel 5 – Operatoren

<- Kapitel 4 · Inhalt · Weiter: Kapitel 6

Übersicht

Kategorie	Operatoren	Nativ
Arithmetisch	+ - * / % (binär), - (unär)	ja
Vergleich	== != < > <= >=	ja
Logisch	&& || !	ja (kurzschließend)
Bitweise	& | ^ << >>	ja
Bitweises NOT	~	nein (NX2002)
Zuweisung	= += -= *= /= %=	ja
Weitere Zuweisungen	&= |= ^= <<= >>=	Frontend
Inkrement/Dekrement	++ -- (prä/post)	nein (NX2002); stattdessen a += 1
Ternär	bedingung ? a : b	ja
Sonstige Token	. , ; : :: -> => ( ) [ ] { }	Syntax

Rangfolge (hoch -> niedrig), Assoziativität

Stufe	Operatoren	Assoziativität
11	* / %	links
10	+ -	links
9	<< >>	links
8	< <= > >=	links
7	== !=	links
6	&	links
5	^	links
4	|	links
3	&&	links
2	||	links
2/1	?: (Ternär)	rechts
1	= und zusammengesetzte Zuweisungen	rechts

Unäre Operatoren (- ! ~) binden stärker als alle binären. Quelle: Parser (compiler/src/parser/mod.rs).

Verhalten

```
public int main() {
    int a = 17;
    return a % 5 + (-a) / 4;     // 2 + (-4) = -2 (Division schneidet Richtung 0 ab)
}
```

- Ganzzahldivision rundet zur Null: -7 / 2 ergibt -3.
- &&/|| werten rechts nur bei Bedarf aus: a != 0 && 10 / a > 1 ist sicher bei a == 0.
- Overflow: Standard ist Trap. add(2147483647, 1) beendet das Programm mit Exitcode -1073741795 (0xC000001D). Mit --overflow-checks=false wird umlaufend gerechnet. Division durch 0 trappt ebenfalls.

Typkompatibilität

Beide Operanden eines arithmetischen/Vergleichsoperators müssen denselben numerischen Typ haben (NX1102). Bedingungen müssen bool sein. Überladbar sind nur == != = (static, zwei Parameter, Rückgabe bool; nur Frontend).

Häufige Fehler

- a++ im nativen Code -> NX2002 Operator 'post++' wird im aktuellen MIR-Subset nicht unterstützt.
- Mischen von int und uint/long ohne Cast-Möglichkeit.

Operatorüberladung

Überladbar sind ausschließlich ==, !=, , =. Jede Überladung ist static, hat genau zwei Parameter und liefert bool:

```
public class Vector3
{
    public static bool operator ==(Vector3 left, Vector3 right)
    {
        return left.x == right.x && left.y == right.y && left.z == right.z;
    }
}
```

Das Frontend prüft Operatorliste, static, Parameterzahl und bool-Rückgabe. Geplant: Prüfung der Parameterbeziehung zum definierenden Typ und der Konsistenz (==/!=, sinnvolle Ordnung); der Compiler kann mathematische Konsistenz beliebiger Implementierungen nicht beweisen.

Nicht überladbar: + - * / %, & | ^ ~, ! && ||, >, ++ --, Indexierung, Aufruf, Memberzugriff, Zuweisung und eigene Operatorzeichen. && und || behalten Kurzschlussauswertung. ? gehört nur zum Ternäroperator c ? a : b; einen ?-Fehlerfortleitungsoperator gibt es nicht.

Kapitel 6 – Kontrollstrukturen

<- Kapitel 5 · Inhalt · Weiter: Kapitel 7

Alle Strukturen dieses Kapitels sind in examples/language/03_conditions.nx und 04_loops.nx verifiziert und laufen nativ.

if / else

```
if (bedingung) { ... } else if (bedingung) { ... } else { ... }
```

Die Bedingung steht in Klammern und muss bool sein. Geschweifte Klammern dürfen bei einer einzelnen Anweisung entfallen (if (a == 1) return 5; getestet).

```
public int classify(int score) {
    if (score >= 90) {
        return 1;
    } else if (score >= 50) {
        return 2;
    }
    return 3;
}
```

Verschachtelung ist beliebig (nested_if getestet). Häufiger Fehler: Nicht-bool-Bedingung -> NX1102.

while

```
while (bedingung) { ... }
```

Prüft vor jedem Durchlauf. while (true) mit break ist zulässig. Es gibt kein do … while.

for

```
for (mut int i = 0; i < 10; i += 1) { ... }
```

Klassische Form: Initialisierung; Bedingung; Fortschritt. Die Laufvariable muss mut sein (sonst NX1200). i++ als Fortschritt scheitert nativ; i += 1 verwenden. foreach / for (T v in seq) ist nur im Frontend vorhanden.

break und continue

break verlässt die innerste Schleife, continue springt zur nächsten Iteration. Außerhalb: NX1301 (break) bzw. NX1302 (continue).

return

return ausdruck; beendet die Funktion. In Funktionen mit Rückgabetyp ist ein Wert Pflicht (return; -> NX1103); jeder Kontrollpfad muss zurückkehren (NX1103 Nicht jeder Kontrollfluss … liefert einen Rückgabewert).

Ternär

int t = sum > 5 ? 1 : 2; – rechts-assoziativ, nativ verfügbar.

Weitere Konstrukte (nur Frontend)

switch/case/default -> im Build NX2001 Diese Kontrollflussanweisung wird in der aktuellen Lowering-Stufe nicht unterstützt; try/catch -> ebenso; match ist teilweise geparst (Armsyntax im Test nicht akzeptiert, NX0100).

Kapitel 7 – Funktionen und Methoden

<- Kapitel 6 · Inhalt · Weiter: Kapitel 8

Funktionsdefinition

```
[Modifier] Rückgabetyp name(Typ param1, Typ param2) { Rumpf }

public int add3(int a, int b, int c) {
    return a + b + c;
}
```

- Modifier: public, private, static sind nativ getestet (private int helper(), public static int twice(int n)).
- Parameter sind unveränderlich (a = a + 1; -> NX1200); mut-Parameter existieren nicht (NX0100 Typname erwartet).
- Jede Funktion mit Nicht-void-Rückgabetyp braucht einen Rumpf und auf allen Pfaden ein return (NX1103). Eine Signatur ohne Rumpf -> NX1005.

Aufruf, Rekursion

Aufrufe sind nativ nur direkte Aufrufe freier, nicht-generischer Funktionen. Reihenfolge der Definitionen ist egal (wechselseitige Rekursion even/odd getestet).

```
public int fib(int n) {
    if (n < 2) { return n; }
    return fib(n - 1) + fib(n - 2);
}
```

Falsche Argumentzahl: NX1107 Keine Überladung von 'f' akzeptiert 2 Argumente. Kandidaten: f(int). Unbekannte Funktion: NX1004.

Rückgabewerte

Genau ein Wert über return. String-Literale können lokalen Variablen zugewiesen, als Parameter übergeben und von internen Funktionen zurückgegeben werden. void-Funktionen sind im nativen Build nicht aufrufbar, außer Console.write/writeLine (NX2002 Void-Aufrufe sind im MIR-Subset nur für Console.write und Console.writeLine verfügbar). String-Parameter/Rückgaben sowie bool, int, uint werden nativ unterstützt; String-ABI in extern "C" ist nicht unterstützt.

Lokale Variablen

Gelten ab Deklaration bis Blockende, sind pro Aufruf getrennt (Kapitel 4).

Methoden und Sichtbarkeit (nur Frontend)

Methoden stehen in class/struct; public/private/protected/internal werden geparst. Aufruf c.inc() besteht check, der Build lehnt Klassen/new ab (NX2001; siehe Kapitel 8). Konstruktoren heißen wie der Typ; die Signatur- und Überladungsprüfung fehlt.

Einschränkungen

Keine Generics, Lambdas, Überladungsauflösung nativ, Funktionszeiger oder Standardparameter.

Entwurfsregeln und Frontend-Stand

- Die C#-ähnliche Deklaration (public int add(int a, int b)) ist die einzige Form; fn name(...) -> T gehört nicht zur Sprache. Konstruktoren heißen wie der Typ und haben keinen Rückgabetyp.
- Generische Parameter stehen hinter dem Namen: public T identity(T value).
- Überladungen unterscheiden sich durch Parametertypen, nicht allein durch den Rückgabetyp. Unsuffigierte Integer-Literale bevorzugen bei sonst gültigen Überladungen ganzzahlige vor Fließkomma-Parametern; gleichrangige Kandidaten bleiben mehrdeutig (NX1107).
- Das Frontend erkennt fehlende Rückgabepfade, generische Methodenaufrufe und Typargument-Substitution in einfachen Signaturen; ein nicht-statischer Methodenaufruf über den Typnamen wird abgelehnt.
- Geplant: vollständige generische Inferenz, Constraint-Prüfung, modulübergreifende Auflösung, Codegenerierung.

Kapitel 8 – Structs und Datenstrukturen

<- Kapitel 7 · Inhalt · Weiter: Kapitel 9

> Status: nur Frontend. struct, class, new, Feldzugriff und Methoden bestehen nortexc check, aber nortexc build meldet NX2001 Dieser Ausdruck wird in der aktuellen Lowering-Stufe nicht unterstützt. Sie erzeugen keine EXE. Verifiziert mit 07_struct_frontend.nx und 08_class_frontend.nx.

Struct-Definition

```
public struct Point {
    public int x;
    public int y;
}
```

Felder besitzen Typ und Sichtbarkeit; public int value = 0; erlaubt einen Initialwert (Klassenfeld, getestet).

Initialisierung und Zugriff

```
mut Point p = new Point();
p.x = 3;
p.y = 4;
```

Eine Zuweisung an ein Feld einer unveränderlichen Bindung wird abgelehnt: NX1201 Die Veränderung über ein Member oder einen Index erfordert einen exklusiven &mut-Borrow. Deshalb mut Point p.

Klassen mit Methoden

```
public class Counter {
    public int value = 0;
    public void inc() { value = value + 1; }
    public int get() { return value; }
}
```

Konstruktor public Counter(int x) { this.v = x; } wird geparst; Zuweisungen an Felder scheitern im Lowering (NX2001 Zuweisungen sind nur an lokale Namen … zulässig).

Weitere Deklarationen

record, interface, trait, enum (public enum Color { Red, Green }) werden geparst. Color c = Color.Red; scheitert nativ. Vererbung : Base, IContract wird angenommen, Vertragsprüfung fehlt. Generische Typen und where-Klauseln sind nicht unterstützt.

Arrays und Sammlungen

Array-Typen T[], [T; N] werden in Signaturen geparst; Array-Erzeugung new T[n] fehlt (NX0100). Nativ sind Arrays nicht erlaubt (NX2002 Parametertyp 'int[]' liegt außerhalb des skalaren MIR-Subsets). Collections liegen nur als .nx-Entwürfe in standard-library/collections/ vor.

Verschachtelung

Verschachtelte Strukturen sind im Typmodell vorgesehen (type_system/struct_layout.rs berechnet Layouts), aber nicht ausführbar.

Klassen, Vererbung, Generics, Collections (Zielentwurf)

Parser und erste Deklarationsprüfungen sind vorhanden; Konstruktorauflösung, Receiver-Borrowing, vollständige Vererbung, Interface-/Trait-Prüfung und Trait-Coherence fehlen. Alles Folgende ist geplant:

- Klasse: nominaler Typ mit Identität; new erzeugt eine Instanz mit genau einem Owner. Zuweisung/Übergabe verschiebt den Owner (kein Kopieren); Teilen über Rc/Arc. Der Receiver-Vertrag von Instanzmethoden ist offen (Kapitel 15).
- Struct: Werttyp, Move ist Standard; Kopieren nur mit nachgewiesener Copy-Eigenschaft; Heap nur explizit über Box o. Ä.
- Record: unveränderlicher Werttyp mit struktureller Gleichheit (Feldtypen brauchen Gleichheit); erbt nicht von Klassen.
- Interface: reine Signaturen, keine Felder, keine Standardimplementierungen. Trait: Fähigkeitsvertrag mit Standardmethoden und assoziierten Typen. Beide stehen in der :-Liste des Typs; impl Trait for Type gibt es nicht.
- Vererbung: höchstens eine Basisklasse, beliebig viele Interfaces/Traits; keine Mehrfachvererbung von Klassen; Structs/Records erben nicht von Klassen.
- Generics: Container, identity; erste Strategie ist Monomorphisierung mit Deduplizierung gleicher Instanziierungen. Bounds mit where T: Drawable (mehrere mit +); pro Paket nur eine Implementierung je Trait/Typ-Paar, und Trait oder Typ muss im eigenen Paket liegen (Kohärenz). Ist: der Parser kennt generische Parameter, die Semantik prüft nur die Argumentanzahl bekannter Typen.
- Collections (System.Collections, nur API-Signaturen in standard-library/collections/*.nx): List (komfortabel, amortisiertes O(1)-Append), Vec (Low-Level, explizite Kapazität; keine implizite Umwandlung zu List), Dictionary, HashSet, Queue, Stack. Längen und Indizes sind usize, lesender Zugriff liefert Option, verändernder Option. Speicherverwaltung, Hashing und Iteratoren fehlen.

Kapitel 9 – Speicherverwaltung

<- Kapitel 8 · Inhalt · Weiter: Kapitel 10

Tatsächlicher Stand

Native NORTEX-Programme arbeiten aktuell ausschließlich mit Skalaren auf dem Stack bzw. in Registern. Es gibt keine Heap-Allokation, keine Freigabe und keine Pointer, die ein Programm schreiben könnte.

Thema	Stand
Stack	Locals und Parameter skalarer Typen; Rekursion nutzt den Systemstack (Verhalten bei Stacküberlauf nicht dokumentiert/geprüft).
Heap / new	Frontend akzeptiert new T(); Build: NX2001. Kein Allokator angebunden.
Freigabe / Destruktoren	nicht implementiert (Destruktoren: nicht unterstützt)
Pointer / unsafe	unsafe { } wird nicht geparst; Raw Pointer fehlen
Box, Rc, Arc	nur in Entwurfsdokumenten (ownership.md)
Garbage Collector	bewusst keiner

Sprachziel: Ownership

Das Zielmodell (ownership.md): ein Owner pro Wert, Move bei Übergabe, &T/&mut T Borrows, explizites clone(). Der Compiler prüft davon teilweise (MIR/Semantik): Moves aus lokalen Bindungen, Konflikte zwischen &/&mut, &mut aus unveränderlicher Bindung (NX1200/NX1201). Vollständige Lifetime-/Regionsinferenz fehlt.

Sicherheitsmechanismen (implementiert)

- Unveränderlich als Default; Mutation nur über mut.
- Kein null in sicherem Code.
- Overflow- und Division-durch-null-Trap (Exitcode -1073741795).
- Skalare sind Copy; keine Aliasing-Probleme möglich.

Lebensdauer

Lokale Variablen leben bis zum Ende ihres Blocks bzw. Funktionsaufrufs.

Runtime

runtime/ enthält Rust-Prototypen (allocation, memory, exceptions, async); sie sind nicht an den Compiler gebunden. Gebunden ist nur die Console-Ausgabe (siehe Kapitel 10).

Geplantes Speichermodell (Entwurf)

Kein Garbage Collector; Ownership und Borrowing: Moves übertragen den Owner, &T erlaubt geteilte lesende, &mut T einen exklusiven schreibenden Borrow, keine Referenz überlebt ihren Owner. Raw-Pointer nur in unsafe. Span ist an eine Borrow-Lifetime gebunden und darf nicht über den Speicher hinaus gespeichert werden; Memory besitzt seinen Speicher. Speicher wird über den

Allocator freigegeben, der ihn bereitgestellt hat. Vollständige Move-/Borrow-/Lifetime-Prüfung ist nicht implementiert; Details: ownership.md.

Kapitel 10 – Ein- und Ausgabe

<- Kapitel 9 · Inhalt · Weiter: Kapitel 11

Konsolenausgabe

```
Console.write("Text ohne Zeilenumbruch");
Console.writeLine("Text mit Zeilenumbruch");
```

Beide Aufrufe akzeptieren im nativen Build genau einen Wert vom Typ string. String-Literale und String-Variablen können ausgegeben werden. Die Runtime erhält den UTF-8-Datenzeiger und die Byte-Länge über einen ABI-Aufruf; dies ist keine allgemeine C-ABI-String-Anbindung.

```
using System;

public int main()
{
    string greeting = "Hallo";
    Console.write(greeting);
    Console.writeLine(" NORTEX");
    return 0;
}
```

Interne Stringwerte verwenden einen 16-Byte-Slice-Deskriptor (Datenzeiger und Byte-Länge), der als Adresse gemäß Windows-x64-ABI übergeben wird. String-Literalbytes und Deskriptoren liegen schreibgeschützt im COFF-Datenbereich. Leere Strings und eingebettete NUL-Bytes werden anhand der Länge korrekt behandelt.

```
using System;

public int main() {
    Console.write("a");
    Console.writeLine("b\tc \"zitiert\" \\");
    return 0;
}
```

Ausgabe: ab?c "zitiert" \. Escapes: \n \r \t \0 \" \\. Ein leerer String ist erlaubt. Unicode im Literal wird als UTF-8-Bytes ausgegeben; die korrekte Darstellung hängt von der Konsolen-Codepage ab (nicht systematisch geprüft).

Nicht unterstützt (jeweils NX2001/NX2002)

Versuch | Meldung

Console.writeLine(5); | Argument muss vom Typ string sein

Console.readLine() | nur direkte freie Funktionen und Console-Ausgabe

String-Konkatenation

Console.writeLine("Hallo " + name); ist erlaubt: + zwischen zwei string-Werten erzeugt über nortex_string_concat (Windows HeapAlloc) einen neuen Heap-String. Dieser wird beim Scope-Ende über nortex_string_free freigegeben (Drop im Codegen, siehe 04-data-types.md). Top-Level-Strings (const string/mut string) und extern "C"-Aufrufe mit string-Argument (char*, NUL-terminiert) sind ebenfalls möglich, siehe 04-data-types.md.

Konsoleneingabe, Formatierung und Interpolation sind nicht implementiert. Fehlerausgabe (stderr) hat keine eigene API.

Ergebniswerte als Ausgabekanal

Weil Zahlen nicht ausgegeben werden können, liefern Beispielprogramme Ergebnisse als Exitcode (return countPrimes(100); -> 25). In PowerShell: $LASTEXITCODE.

Standardbibliothek

standard-library/system/console.nx und io/mod.nx sind Entwürfe ohne Compilerbindung.

Strings (Zielentwurf)

string ist UTF-8 und besitzt seinen Speicher. length() zählt Bytes (nicht Skalarwerte oder Graphemcluster); Indexzugriff ist byte-orientiert und liefert byte; chars() iteriert Unicode-Skalarwerte (char, 32 Bit), bytes() die UTF-8-Bytes. Strings müssen gültiges UTF-8 bleiben. Vorgesehene API: length(), isEmpty(), contains(), startsWith(), endsWith(), replace(), split(), trim(), toUpperCase(), toLowerCase(), chars(), bytes(). Ein öffentlicher StringBuilder ist nicht vorgesehen. Der Typchecker akzeptiert string + string und string + skalare Primitive (Ergebnis string); die Formatierung ist noch nicht End-to-End verfügbar.

Kapitel 11 – Fehlermeldungen und Debugging

<- Kapitel 10 · Inhalt · Weiter: Kapitel 12

Format

```
error[NX1102]: Typfehler in Variableninitialisierung: erwartet 'float', erhalten 'double'.
 --> datei.nx:7:19
   |
 7 |     float f = 2.5;
   |               ^^^
```

Meldungen sind deutsch; Codes (NXnnnn) sind stabil. Severity: error oder warning. nortexc bricht bei Fehlern mit Exitcode != 0 ab und schreibt keine EXE.

Fehlerklassen mit tatsächlich beobachteten Codes

Klasse | Code | Beispiel / Behebung

Lexikalisch/Syntax | NX0100 | int a = ; · @ · fehlendes ; · nicht abgeschlossener String · unbekannte Escape \q. Syntax korrigieren.

Doppelte Deklaration | NX1001 | int a zweimal im Scope -> umbenennen.

Namensauflösung | NX1003, NX1004 | Unbekannter Namespace/Bezeichner -> deklarieren oder korrigieren.

Memberdeklaration | NX1005 | Funktion ohne Rumpf -> { ... } ergänzen.

Typen | NX1100, NX1102 | int a = "text"; -> passenden Typ nutzen; Literal außerhalb des Wertebereichs.

Rückgabe | NX1103 | Nicht alle Pfade geben zurück -> return ergänzen.

Überladung/Aufruf | NX1107 | Falsche Argumentzahl.

Veränderlichkeit | NX1200, NX1201 | Zuweisung an unveränderliche Bindung -> mut setzen.

Kontrollfluss | NX1301, NX1302 | break/continue außerhalb einer Schleife.

Stil-Warnung | NX0004 | Konstante nicht UPPER_SNAKE_CASE, Variable nicht camelCase.

Backend-Subset | NX2001, NX2002 | Konstrukt vom nativen Lowering nicht unterstützt (siehe unten).

Codegen/Linker | (ohne Code) | „genau eine parameterlose int-main-Funktion“, LNK2019 bei fehlender main.

> Abweichung zur Codetabelle: diagnostic-codes.md reserviert NX0001–NX0003 für Lexerfehler. Der aktuelle Lexer meldet diese Fälle tatsächlich als NX0100. Maßgeblich ist das beobachtete Verhalten.

Laufzeitfehler

Es gibt keine Laufzeit-Fehlermeldungen oder Exceptions. Ein Overflow- oder Division-durch-0-Trap beendet das Programm mit Exitcode -1073741795 (0xC000001D, illegale Instruktion) ohne Text. Mit --overflow-checks=false entfällt der Overflow-Trap.

Bekannter Compilerfehler

Programme mit long-Locals und Division/Vergleich gegen int-Literal können einen Panic im Cranelift-Frontend auslösen (frontend.rs), statt einer Diagnose. Workaround: long nur als Parameter/Rückgabe verwenden.

Strategien

1. nortexc check datei.nx für reine Frontend-Fehler.
2. NX2001/NX2002 bedeuten „noch nicht nativ“, nicht „falscher Code“: auf unterstützte Konstrukte umstellen (README).
3. nortexc mir datei.nx zeigt das optimierte MIR.
4. Ergebnisse über den Exitcode ausgeben (Kapitel 10).

Fehlerbehandlung (Entwurf und Frontend-Stand)

NORTEX trennt optionale Werte (Option: Some/None), erwartbare Fehler (Result: Ok/Err) und Exceptions (throw, try/catch/finally). Es gibt keinen ?-Operator. Option und Result sollen must_use sein; ein nicht verwendetes Ergebnis muss behandelt, weitergegeben oder mit discard(value) verworfen werden.

```
public Result<int, string> divide(int first, int second)
{
    if (second == 0) { return Err("Division by zero"); }
    return Ok(first / second);
}
```

Ist-Stand (Frontend): Ein Result als eigenständiges Ausdrucksstatement meldet NX1108; nicht vollständige Enum-match-Arme melden NX1109 (_ ist der Fallback). Option wird nicht auf must_use geprüft. Geplant: discard, native Exception-Unwinding, RAII-Bereinigung, vollständiges match, Exceptions als Statuswerte an der C-ABI-Grenze; checked/unchecked ist offen (Kapitel 15).

Asynchronität (geplant)

Ziel: async-Funktionen werden zu suspendierbaren Zustandsmaschinen; await blockiert keinen Worker. Future ist die Berechnung, Task die von der Runtime eingeplante Ausführung.

```
public async Task<string> loadText(string path)
{
    return await File.readText(path);
}
```

Der Parser kennt async/await; der Compiler senkt sie nicht in die Runtime ab. Der Rust-Prototyp runtime/src/async/ hat einen Waker-basierten Scheduler mit FIFO-Queue und block_on (nur für Einstieg/Tests). Es fehlen Work-Stealing, Timer, Cancellation-API, IOCP-Reaktor und Send/Sync-Analyse.

Kapitel 12 – Kompilierung und technische Funktionsweise

<- Kapitel 11 · Inhalt · Weiter: Kapitel 13

Pipeline

```
.nx -> Lexer -> Parser/AST -> semantische Prüfung -> HIR -> MIR
    -> MIR-Validierung + Optimierung -> Cranelift -> Windows-x64-COFF (.obj)
    -> MSVC-Linker (+ NORTEX-Runtime) -> .exe
```

Stufe | Quelle | Aufgabe

Lexer | compiler/src/lexer/mod.rs | ASCII-Bezeichner, Schlüsselwörter, Zahlen, Strings/Chars, Symbole, Kommentarspannen (für die IDE)

Parser/AST | parser/mod.rs, ast/mod.rs | Pratt-Parser für Ausdrücke, Deklarationen, Anweisungen; Fehler als NX0100

Semantik | semantic/mod.rs, type_checker.rs | Namen, Typen, Veränderlichkeit, Rückgabepfade, Move/Borrow-Grundprüfungen, Stilwarnungen

HIR | hir/mod.rs | aufgelöste, typisierte Zwischenform

MIR | mir/lowering.rs, mir/mod.rs | CFG mit Blöcken; skalares Subset; Definite-Assignment- und Loan-Prüfung; Diagnosen NX2001/NX2002 für nicht unterstützte Konstrukte

Optimizer | optimizer/mod.rs | Optimierungen auf validiertem MIR

Codegen | codegen/mod.rs | Cranelift-IR -> COFF-Objekt für x86_64-pc-windows-msvc; Overflow-Trap

Linker | linker/mod.rs | ruft MSVC link.exe, bindet CRT und Runtime; zusätzliche Argumente aus dem Manifest

Driver | driver/mod.rs | verbindet die Stufen

Kommandos

Kommando | Wirkung

nortexc check | Lexer + Parser + Semantik

nortexc mir | gibt validiertes, optimiertes MIR aus

nortexc build [-o …] [--emit-obj] [--overflow-checks[=bool]] | erzeugt .exe oder .obj

Eigenschaften des Backends

- Ziel: ausschließlich Windows x64 (MSVC). Kein Linux/ARM64.
- Einstieg: C-main der CRT; NORTEX-int main() liefert den Prozess-Exitcode.
- Stringwerte werden als Adresse eines 16-Byte-UTF-8-Slice-Deskriptors repräsentiert; Console.write(Line) lädt daraus Datenzeiger und Byte-Länge für den Runtime-ABI-Aufruf.
- Nicht unterstützte Konstrukte werden diagnostiziert, nicht still ignoriert (Ausnahme: im getesteten Fall async-Deklaration baut ohne Wirkung; enum/class-Deklarationen ohne Nutzung werden akzeptiert).

Hinweis zu älteren Dokumenten

docs/compiler/architecture.md beschreibt einen früheren Stand („natives Backend noch nicht fertig“). Der native Pfad für das skalare Subset ist inzwischen vorhanden und durch End-to-End-Tests belegt (sum_to(10) -> Exitcode 55).

ABI (Zielentwurf)

extern "C" markiert eine Funktion mit C-ABI; nur ABI-kompatible Parameter/Rückgaben sind zulässig. Structs sind nur mit [repr(C)] C-kompatibel; string, Klassen-Owner, Generics, Exceptions, Task/Future und beliebige Result/Option-Layouts haben keine automatische C-Kompatibilität. Strings gehen als UTF-8-Zeiger plus Byte-Länge; Speicher wird vom bereitstellenden Allocator freigegeben. Symbolnamen, Calling Convention und ABI-Hauptversion gehören zum Schnittstellenvertrag. Eine freigegebene Binärkompatibilitätsgarantie und ein FFI-Testlauf fehlen; Verbindliches steht in Phase 4.

Paketformat .ntx (geplant)

.nx ist Quelltext, nortex.toml das TOML-Manifest, .ntx ein eigener deterministischer Binär-Container (kein ZIP, kein Quelltext). Reader und Writer sind nicht implementiert. Entwurf (Containerformat 1, little-endian): 48-Byte-Header mit Magic NORTEXPK, Major-/Minorversion, Flags (unbekannte Pflichtflags -> Ablehnung), Offset/Länge des TOML-Manifests, Index-Offset, Eintragszahl und reservierten Nullbytes; Index je Datei mit Pfadlänge, Typ, Flags, Offsets, Größen und SHA-256, lexikografisch sortiert; 8-Byte-ausgerichtete Nutzdaten, keine Kompression. Validierung: nur relative UTF-8-Pfade mit / (kein .., keine Duplikate), alle Bereiche innerhalb der Datei, Größenlimits vor Allokation, Integerüberlauf-Prüfung, SHA-256-Prüfung (Schutz vor Beschädigung, keine Authentifizierung), optionale Signaturen (Verfahren offen), atomare Installation ohne Symlinks. Builds sollen byteidentisch reproduzierbar sein.

Kapitel 13 – Praktische Beispiele

<- Kapitel 12 · Inhalt · Weiter: Kapitel 14

Alle Dateien liegen in examples/language/. Jedes Beispiel wurde mit nortexc build gebaut und ausgeführt (Ausnahmen: 07/08, nur check). Aufruf: nortexc build examples\language\NN_name.nx -o out.exe; .\out.exe; $LASTEXITCODE.

1. Hello World – 01_hello.nx

```
using System;

public int main() {
    Console.writeLine("Hallo, NORTEX!");
    return 0;
}
```

Gibt Hallo, NORTEX! aus, Exitcode 0. main ist der Einstieg, der return-Wert der Exitcode.

2. Variablen und Berechnungen – 02_variables.nx

```
public int main() {
    int width = 6;
    int height = 7;
    int area = width * height;
    mut int total = area;
    total += 8;
    total -= 3;
    const int OFFSET = 3;
    return total + OFFSET;
}
```

Unveränderliche Werte per Default, mut für total. Rechnung: 42 -> 50 -> 47 -> 50. Exitcode 50.

3. Bedingungen – 03_conditions.nx

```
public int classify(int score) {
    if (score >= 90) {
        return 1;
    } else if (score >= 50) {
        return 2;
    }
    return 3;
}

public int main() {
    Console.writeLine("Klassifiziere den Wert 72");
    return classify(72);
}
```

72 erfüllt nur die zweite Bedingung. Ausgabe Klassifiziere den Wert 72, Exitcode 2.

4. Schleifen – 04_loops.nx

```
public int main() {
    mut int sum = 0;
    for (mut int i = 1; i <= 10; i += 1) {
        if (i % 2 != 0) {
            continue;
        }
        sum += i;
    }

    mut int n = 0;
    while (true) {
        n += 1;
        if (n == 5) {
            break;
        }
    }
    return sum + n;
}
```

for summiert gerade Zahlen (2+4+6+8+10 = 30), continue überspringt ungerade; while (true) endet per break bei 5. Exitcode 35.

5. Funktionen und Rekursion – 05_functions.nx

```
public int gcd(int a, int b) {
    if (b == 0) {
        return a;
    }
    return gcd(b, a % b);
}

public int fib(int n) {
    if (n < 2) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

public int main() {
    return gcd(48, 18) + fib(10);
}
```

gcd(48,18) = 6, fib(10) = 55. Exitcode 61.

6. Vollständige Anwendung: Primzahlen – 06_primes.nx

```
using System;

public bool isPrime(int n) {
    if (n < 2) {
        return false;
    }
    mut int d = 2;
    while (d * d <= n) {
        if (n % d == 0) {
            return false;
```

```
        }
        d += 1;
    }
    return true;
}

public int countPrimes(int limit) {
    mut int count = 0;
    for (mut int n = 2; n <= limit; n += 1) {
        if (isPrime(n)) {
            count += 1;
        }
    }
    return count;
}

public int main() {
    Console.writeLine("Zaehle Primzahlen bis 100");
    return countPrimes(100);
}
```

Kombiniert bool-Funktionen, Schleifen, frühe Rückgaben, Aufrufketten und Konsolenausgabe. Es gibt 25 Primzahlen <= 100. Exitcode 25.

7. Structs – 07_struct_frontend.nx (nur check)

```
public struct Point {
    public int x;
    public int y;
}

public int main() {
    mut Point p = new Point();
    p.x = 3;
    p.y = 4;
    return p.x + p.y;
}
```

nortexc check erfolgreich; nortexc build -> NX2001. Zeigt die vorgesehene Syntax, ist aber noch nicht ausführbar.

8. Methoden – 08_class_frontend.nx (nur check)

Klasse Counter mit inc()/get(); gleicher Status wie Beispiel 7 (siehe Kapitel 8).

9. Heap-Allokation

Nicht ausführbar: new erzeugt im nativen Build immer NX2001. Es gibt daher kein lauffähiges Heap-Beispiel (Kapitel 9).

Kapitel 14 – Sprachreferenz

<- Kapitel 13 · Inhalt · Weiter: Kapitel 15

Schlüsselwörter

Quelle: is_keyword in compiler/src/lexer/mod.rs.

Gruppe | Wörter | Nativ nutzbar

Deklarationen | using namespace class struct record interface trait enum | nur Deklaration akzeptiert

Modifier | public private protected internal static abstract sealed partial extern unsafe const virtual override async readonly | public private static const

Veränderlichkeit | var mut ref out params | mut

Kontrollfluss | return throw if else while for in switch case default break continue try catch finally match | return if else while for break continue

Literale | true false null | true false (null verboten)

Ausdrücke | new await this base operator is as typeof sizeof where | nein

Reserviert | let | nein (NX0100)

Typen | sbyte short int long i128 byte ushort uint ulong u128 nint nuint isize usize float double decimal bool char string void | int bool uint short ulong long (Einschränkungen: Kap. 4)

Nicht-Schlüsselwörter: Console, System, get, set, do, foreach, fn.

Token

Identifier, Keyword, Number (dezimal, 0x, 0b, Exponent, Suffix), StringLiteral, CharacterLiteral, Symbol, EndOfFile. Kommentare werden separat erfasst.

Operatoren und Symbole

+ - * / % · == != = · && || ! · & | ^ ~ > · = += -= *= /= %= &= |= ^= >= · ++ -- · ? : · . , ; ( ) [ ] { } · ->
=> ::. Rangfolge: Kapitel 5.

Grammatikalische Konstruktionen (nativ)

```
Datei        = { Using } { Funktion } ;
Using        = "using" Name ";" ;
Funktion     = { Modifier } Typ Name "(" [ Typ Name { "," Typ Name } ] ")" Block ;
Block        = "{" { Anweisung } "}" ;
Anweisung    = Deklaration | Zuweisung ";" | Aufruf ";" | If | While | For
             | "break" ";" | "continue" ";" | "return" [ Ausdruck ] ";" | Block ;
Deklaration  = [ "mut" | "const" ] Typ Name "=" Ausdruck ";" ;
If           = "if" "(" Ausdruck ")" Anweisung [ "else" Anweisung ] ;
While        = "while" "(" Ausdruck ")" Anweisung ;
For          = "for" "(" Deklaration Ausdruck ";" Zuweisung ")" Anweisung ;
Ternär       = Ausdruck "?" Ausdruck ":" Ausdruck ;
```

Das ist eine vereinfachte Sicht des nativ unterstützten Teils. Die vollständige Frontend-Grammatik: grammar.ebnf.

Unterstützte Anweisungen

Lokale Deklaration, Zuweisung (einfach/zusammengesetzt), Funktionsaufruf, Console.write(Line), if/else, while, for, break, continue, return, Block.

Built-in-Funktionen

Funktion	Signatur	Einschränkung
Console.write	(string)	genau ein Stringwert
Console.writeLine	(string)	genau ein Stringwert

Weitere eingebaute Funktionen existieren nicht (Console.readLine nicht auflösbar).

Diagnosecodes (beobachtet)

NX0004 NX0100 NX1001 NX1003 NX1004 NX1005 NX1100 NX1102 NX1103 NX1107 NX1200 NX1201 NX1301 NX1302 NX2001 NX2002. Siehe Kapitel 11.

Relevante Einschränkungen

intrinsic ist ein kontextabhängiger Modifier ausschließlich für Dateien unter standard-library/; in Anwendungscode wird er mit NX1005 abgelehnt. Ein Intrinsic-/Runtime-Symbolbinder existiert nicht.

Siehe Feature-Übersicht. Zusätzlich: nur Windows x64; main muss public int main() ohne Parameter sein; Modulsystem ohne qualifizierte Namen und Sichtbarkeit (using mit flachem Namensraum); long-Locals mit Division können den Codegen abstürzen lassen.

Kapitel 15 – FAQ und Best Practices

<- Kapitel 14 · Inhalt

Häufige Anfängerfehler

Fehler	Ursache / Lösung
NX1200 bei i += 1 in for	Laufvariable mut deklarieren: for (mut int i = 0; …)
NX1200 bei Parameter ändern	Parameter sind unveränderlich; lokale mut-Kopie anlegen.
NX2001/NX2002 beim Build	Konstrukt ist nur Frontend (Klassen, Heap-Strings, var, switch, …).
„genau eine parameterlose int-main-Funktion“	public int main() verwenden.
a++ scheitert	a += 1;
Keine Zahlenausgabe	Ergebnis als Exitcode zurückgeben.
let x = 1;	Typ angeben: int x = 1;
NX1102 bei uint + int	Gleiche Typen verwenden; es gibt keine Casts.

FAQ

Warum läuft meine Klasse nicht? class/struct/new sind im Frontend, aber nicht im nativen Lowering. Kann ich Strings speichern? Ja. Literale, lokale Variablen, Top-Level-const/mut string, interne Parameter und Rückgaben verwenden UTF-8-Slices; a + b erzeugt Heap-Strings (ohne Freigabe). Wie debugge ich? nortexc check, nortexc mir, Exitcodes; ein Debugger existiert nicht. Gibt es Linux? Nein, nur Windows x64. Gibt es Module? Ja: using util; löst util.nx bzw. util/mod.nx auf (flacher Namensraum), siehe Ist-Spezifikation.

Empfohlene Praktiken

- Namen: Funktionen/Variablen camelCase, Konstanten UPPER_SNAKE_CASE, Typen PascalCase (entspricht den NX0004-Warnungen).
- Veränderlichkeit: Nur dort mut, wo nötig.
- Struktur: Kleine Funktionen mit einem return-Pfad; main nur als Orchestrator.
- Fehlerprüfung: Vor Division auf 0 prüfen (Trap statt Fehlermeldung).
- Typen: Im nativen Code int/bool bevorzugen; explizite Typen statt var.
- Verifikation: Beispiele nach jeder Änderung bauen und Exitcode prüfen.

Performance

Der Backend-Optimierer arbeitet auf MIR; Rekursion (fib) ist nicht optimiert (kein Tail-Call-Nachweis). Overflow-Prüfung kostet etwas; --overflow-checks=false nur bewusst einsetzen, da dann Wraparound ohne Warnung auftritt.

Größere Projekte

Mit nortex.toml bauen (Kapitel 2). Mehrere .nx-Dateien lassen sich über using zusammenführen (flacher Namensraum, keine Sichtbarkeitsprüfung zwischen Modulen; siehe Ist-Spezifikation).

Offene Entscheidungen (Projektinhaber)

Diese Punkte sind nicht freigegeben; der Compiler darf sie nicht als endgültig behandeln:

1. Lizenz: Der Repository-Header lautet MIT; das ist keine bestätigte rechtliche Freigabe.
2. Exceptions: checked oder unchecked und das native Unwind-Verfahren sind offen. Exceptions dürfen nie unkontrolliert über eine C-ABI-Grenze laufen.
3. Arithmetik: Ob Checked-Arithmetik im Release-Standard ist, ist offen (derzeit konfigurierbar über Overflow-Checks, siehe Manifest).
4. Methodenreceiver: &self, &mut self oder ein anderer Vertrag für Instanzmethoden ist offen.
5. Backend: Das Backend ist Cranelift mit MSVC link.exe; LLVM ist nicht verbindlich und höchstens ein nicht-normativer Prototyp.

Frühere Entwurfsregeln, die der Ist-Stand überholt hat (Bindungen „standardmäßig veränderbar“, Manifest .nortex, Linker lld-link, „keine native EXE“), gelten nicht mehr.